Readme

1 — High-level phases (single-line)

1. Landing zone & identity → accounts/projects/subscriptions.
2. Networking (hub-and-spoke) + DNS.
3. Infra-as-code (Terraform modules).
4. Kubernetes (EKS) + Runtime Fabric install.
5. Mule APIs + containerize + Helm.
6. CI/CD pipelines (build → test → deploy).
7. Observability & tracing.
8. MCP adapters & Agentic AI (Order Assistant, AI-Ops).
9. Security + policy + audit.
10. Multi-cloud parity + hardening + docs.

2 — Deliverables per phase (what you'll produce)

- `infra/` Terraform modules for network, EKS, RDS, ECR.
- `helm/` chart for `orders-api` and `notifier`.
- GitHub Actions workflows: `ci.yaml`, `cd.yaml`, `tf-plan-apply.yaml`.
- K8s manifests for Prometheus, Grafana, OpenTelemetry Collector.
- MCP adapters: `mule-adapter`, `terraform-adapter`, `k8s-adapter`, `prom-adapter`.
- `ai-agents/` — two agents: `order-assistant`, `ai-ops-agent`.
- Security policy snippets: OPA/Kyverno, IAM role templates.
- Runbook & playbooks for incident simulation.

3 — Terraform skeleton (minimal, cross-cloud-friendly)

Create `infra/aws/network/main.tf` (example snippet — keep small for clarity):

```
# infra/aws/network/main.tf
provider "aws" {
  region = var.region
}

resource "aws_vpc" "hub" {
  cidr_block = var.vpc_cidr
  tags = { Name = "${var.env}-hub-vpc" }
}

resource "aws_subnet" "private" {
  count             = length(var.private_azs)
  vpc_id            = aws_vpc.hub.id
  cidr_block        = cidrsubnet(aws_vpc.hub.cidr_block, 8, count.index)
  availability_zone = var.private_azs[count.index]
  tags = { Name = "${var.env}-private-${count.index}" }
}
```

Recommended module structure:

```
infra/
  aws/
    network/
      main.tf, variables.tf, outputs.tf
    eks/
    rds/
    ecr/
  gcp/
  azure/
```

```
  global/
    iam/
    dns/
```

State backend example (AWS S3 + DynamoDB locking):

```
terraform {
  backend "s3" {
    bucket         = "orderflow-terraform-state"
    key            = "aws/network/terraform.tfstate"
    region         = "us-west-2"
    dynamodb_table = "orderflow-tf-locks"
  }
}
```

4 — EKS module (core pieces)

`infra/aws/eks/main.tf` simplified:

```
module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  cluster_name    = "${var.env}-eks"
  cluster_version = var.k8s_version
  subnets         = module.network.private_subnet_ids
  vpc_id          = module.network.vpc_id
  node_groups = {
    default = {
      desired_capacity = var.node_desired
      instance_type    = var.node_type
    }
  }
  tags = var.tags
```

```
}
```

Outputs: kubeconfig, cluster_arn, node_role_arn.

5 — Helm chart stub (orders-api)

`helm/orders-api/Chart.yaml` and `values.yaml` basics:

`values.yaml`:

```
replicaCount: 2
image:
  repository: <aws_account>.dkr.ecr.us-west-2.amazonaws.com/orderflow/or
ders-api
  tag: latest
service:
  type: ClusterIP
  port: 8081
ingress:
  enabled: true
  host: orders.dev.orderflow.example.com
```

`templates/deployment.yaml` — basic deployment with liveness/readiness probes. Add service account annotated for IRSA later.

6 — GitHub Actions: CI + CD pattern

`.github/workflows/ci.yaml` (build/test/push):

```
name: CI

on: [push]

jobs:
```

```
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK, Node etc (as required)
      - name: Build Mule app
        run: ./mvnw clean package -DskipTests
      - name: Unit tests
        run: ./mvnw test
      - name: Login to ECR
        uses: aws-actions/amazon-ecr-login@v2
      - name: Build and push image
        run: |
          docker build -t ${{ env.IMAGE }} .
          docker tag ${{ env.IMAGE }} ${{ env.ECR_URI }}:${{ github.sha }}
          docker push ${{ env.ECR_URI }}:${{ github.sha }}
      - name: Create image manifest
        run: echo "image:${{ env.ECR_URI }}:${{ github.sha }}" > image.propertie
s
      - name: Upload image.properties
        uses: actions/upload-artifact@v4
        with:
          name: image
          path: image.properties
```

`.github/workflows/cd.yaml` (deploy via helm + terraform apply):

```
name: CD

on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]

jobs:
```

```
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Download artifact
        uses: actions/download-artifact@v3
        with: name: image
      - name: Terraform Apply (infra)
        uses: hashicorp/terraform-github-actions@v1
        with:
          tf_actions_version: latest
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
      - name: Helm deploy
        uses: azure/setup-helm@v3
      - run: |
          IMAGE=$(cat image.properties)
          helm upgrade --install orders-api helm/orders-api --set image.repositor
y=${IMAGE}
```

7 — Kubernetes: Runtime Fabric + Mule deployment notes

- Install cert-manager, external-dns, alb-ingress-controller (AWS) or GKE ingress.
- RTF typically needs node pools with specific labels; follow MuleSoft instructions for RTF install (you'll run the operator in a namespace).
- Mule runtime as a container image: push Mule runtime image to ECR and use Helm/RTF to manage lifecycle.

8 — MCP tool definitions (example)

Make each tool a small REST wrapper that accepts JSON-structured tool calls. Example `terraform-adapter` POST payload:

```
POST /mcp/terraform/run
{
  "tool": "terraform",
  "action": "plan",
  "workspace": "dev",
  "args": {
    "var": {
      "cluster_size": 3
    }
  },
  "caller": "ai-ops-agent",
  "request_id": "uuid-123"
}
```

Tool response:

```
{
  "status": "ok",
  "result": {
    "plan": "..."
  },
  "request_id": "uuid-123"
}
```

Each adapter should:

- Validate auth token (OIDC).
- Enforce RBAC via a policy engine.
- Return structured logs/events for audit.

9 — Agent design: Python pseudocode (Order Assistant)

Minimal LangChain-like pseudocode (conceptual):

```
# ai-agents/order-assistant/main.py
from mcp_client import MCPClient
from llm_client import LLMClient

mcp = MCPClient(url=os.getenv("MCP_URL"), token=os.getenv("MCP_TOKE
N"))
llm = LLMClient(provider="bedrock", api_key=os.getenv("BEDROCK_KEY"))

def analyze_orders():
    orders = mcp.call_tool("mule-api", "get_recent_orders", {"env":"dev"})
    summary = llm.summarize(orders)
    if detect_fraud(summary):
        mcp.call_tool("mule-api", "flag_orders", {"orders": fraudulent_ids})
    return summary

if __name__ == "__main__":
    schedule.every(5).minutes.do(analyze_orders)
    while True:
        schedule.run_pending()
        sleep(1)
```

AI-Ops agent (reactive):

- Subscribes to alerts (Prometheus Alertmanager webhook).
- Receives `high-latency` event → queries Prometheus via `prom-adapter`, checks pod restarts, runs `k8s-adapter` to roll a restart or scale deployment, logs actions through `audit-adapter`.

10 — MCP adapters list & responsibilities

- `mule-adapter`: call internal Mule endpoints, handle auth.
- `k8s-adapter`: `kubectl` wrapper, maps requested action to safe commands.
- `terraform-adapter`: non-interactive tf runs, respond with plan + apply; requires manual approval gate for prod.
- `prometheus-adapter`: query metrics and return structured results.
- `audit-adapter`: persist every agent action to central audit store (ElastiSearch/CloudWatch Logs).

11 — Guardrails & Policy Examples

OPA policy (deny terraform apply on prod unless tag `allow_ai=true`):

```
package terraform.gate

default allow = false

allow {
  input.action == "apply"
  input.workspace != "prod"
}

allow {
  input.action == "apply"
  input.workspace == "prod"
  input.requester.tags.allow_ai == true
}
```

K8s RBAC (safer pattern):

- `ai-ops` ServiceAccount bound to a Role that only allows `patch` on `deployments` in `orderflow` namespace, not `delete` on PV/DB resources.

12 — Secrets & identity patterns

- K8s: use CSI Secrets Store to mount values from AWS Secrets Manager/GCP Secret Manager/Azure Key Vault.
- Pods get permissions through IRSA (IAM Roles for Service Accounts) on EKS.
- Agents use short-lived tokens fetched from token service (OIDC).

13 — Observability configuration (quick)

- Run OpenTelemetry collector as DaemonSet to gather traces/logs.
- Prometheus scrape config (example job for Mule runtime metrics):

```
scrape_configs:
  - job_name: 'mule'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        regex: orders-api
        action: keep
```

- Grafana: dashboards for p95 latency, error_rate, queue_depth, DB connections, node CPU, pod restarts.

14 — Testing & runbooks

- Unit tests for Mule flows (MUnit).
- Integration tests: a pipeline stage that deploys to `stg` and runs contract tests (Pact) + end-to-end smoke tests.

- Chaos test: Simulate pod eviction and confirm AI-Ops agent triggers remediation.
- Playbooks: “Agent performed scale-up” — who to notify, steps to revert, audit log path.

15 — Minimal viable MCP-enabled scenarios (start here)

1. Agent can read recent orders and produce a summary (no infra changes).
2. Agent can query Prometheus and propose a scaling action (requires PR/approval).
3. Agent can perform a non-destructive Terraform plan and upload plan artifact.
4. Agent can restart a deployment (k8s patch) when safe policy satisfied.

16 — Security checklist (must-do before prod)

- [ ] SSO + MFA enabled.
- [ ] Least-privilege IAM roles for humans & agents.
- [ ] OPA/Kyverno policies enforced on K8s cluster.
- [ ] Image scanning in CI (Trivy).
- [ ] Secrets in SecretsManager/KeyVault, not in repos.
- [ ] Audit logs forwarded to central SIEM.
- [ ] Approvals for destructive Terraform actions in prod.

17 — Data & cost considerations

- Use small instance types for dev. Use autoscaling + Spot for non-prod to reduce cost.
- Tag resources. Set budgets/alerts per cloud account.

18 — 12-week timeboxed plan (practical day-by-day groups)

Weeks 1–2: Landing zone, IAM, Terraform remote state, basic network (hub+spoke).

Weeks 3–4: EKS cluster + cert-manager + ingress + external-dns. Deploy NGINX sample.

Weeks 5–6: Runtime Fabric install + deploy first Mule API (dev). Setup ECR and container pipeline.

Weeks 7–8: CI/CD pipelines + Terraform workflows; automated dev deploys.

Weeks 9–10: Observability (Prometheus/Grafana/OTel) + logging + dashboards.

Weeks 11–12: MCP adapters (mule, prom, k8s), build `order-assistant` agent, test safe actions (non-prod). Harden and write runbooks.

19 — Short example tasks you can do tonight (quick wins)

1. Create a small Terraform `network` plan and run `terraform plan`.
2. Create a Dockerfile for your Mule app and build locally.
3. Create a GitHub repo skeleton with folders above.
4. Spin up a single-node EKS (via Terraform module) and deploy an nginx helm chart.
5. Build a tiny MCP adapter that proxies requests to your Mule API and returns JSON.

20 — Next step (what I can produce now)

I can immediately generate any one of the following starter artifacts — pick one and I'll drop it into the chat (ready-to-copy):

- A **Terraform network + EKS** minimal working repo skeleton (AWS).

- A **GitHub Actions** CI + CD workflow tuned for Mule + Docker + Helm.
- A **Helm chart stub** for your Mule `orders-api`.
- A **MCP tool JSON spec + minimal Node/Python adapter** that forwards agent calls to Mule.
- An **Order Assistant agent** Python skeleton (with pseudo-LLM client + MCP calls).